

Deducing function parameter types using alias template CTAD

Document #: P2998R0
Date: 2024-10-15
Project: Programming Language C++
Audience: Evolution
Reply-to: James Touton
<bekenn@gmail.com>

Contents

- [1 Introduction](#)
- [2 Motivation](#)
- [3 Design](#)
 - [3.1 General approach: CTAD behind the scenes](#)
 - [3.2 Limitations of deduction guides](#)
 - [3.2.1 Declaring deduction guides for alias templates](#)
 - [3.2.2 Alternative: Multiple return types for deduction guides](#)
 - [3.3 Overload resolution](#)
- [4 Wording](#)
- [5 Acknowledgments](#)
- [6 References](#)

§ 1 Introduction

This paper proposes to extend template argument deduction from a function call to include candidates determined via class template argument deduction when the type of a function parameter cannot otherwise be deduced. This will allow some function templates to match a much wider range of invocations involving conversions to the deduced type rather than requiring an exact match. The feature works by leveraging existing wording involving class template argument deduction for alias templates, treating a dependent *template-id* as the aliased type of a hypothetical alias template definition. Template arguments for the hypothetical alias template are deduced using the existing rules for class template argument deduction, and the results are substituted back into the *template-id* of the function parameter type. Template argument deduction for the function template then proceeds as usual from the substituted parameter type.

```

#include <span>
#include <vector>

template <typename T> void f(std::span<T>);
template <typename T> void g(const std::vector<T>&);

void example()
{
    int x[] = { 1, 2, 3, 4, 5 };
    f(x); // previously ill-formed, now OK; T deduced as int
    g({ 1, 2, 3, 4, 5 }); // previously ill-formed, now OK; T deduced as int
}

```

This paper also proposes to allow users to declare deduction guides for alias templates.

§ 2 Motivation

Function templates are intended to behave very much like ordinary functions in that they can be called like ordinary functions and participate in overload resolution with ordinary functions. One way to think of this is that a function template behaves as a collection of overloads consisting of all possible substitutions of its template parameters. At the point of a function call, the implementation compares the list of function arguments against each available overload and selects the overload that most closely matches the function arguments. If the overload originated as a function template, then the template is instantiated using the substitutions that generated the overload.

Of course, that's not how it actually works. The set of substitutions for a template is effectively unbounded; the implementation cannot simply generate a list of all possible substitutions for a template. Instead, the implementation follows a set of rules intended to determine the most appropriate substitution for a template based on the information available in the call and in the structure of the template declaration. This keeps the search space manageable and, importantly, avoids incidental instantiations of other participating templates.

The existing rules are generally very good and have withstood the test of time; function templates are one of the most important bedrock features in the language today, and have been since their inception. The success of the language very much rests on the success of function templates. However, there are limitations to the existing rules for template argument deduction that exclude some desirable use cases.

For the most part, template argument deduction is limited to finding a more-or-less exact match for the type of an argument expression. Conversions are generally not considered. This is usually fine; if, given a type template parameter `T`, a function parameter's type is simply `T` (or some other simple modification, such as `const T&`), then it doesn't make sense to consider substitutions other than the type of the function argument, because any such substitution will be a worse match. Similarly, if a function parameter's type is some variation on `vector<T>` and the argument is a specialization of `vector`, then it is easy (and

correct) to say that the substitution for `T` is the substitution that makes `vector<T>` equal to the argument type.

But there are cases where conversions may be desired, and the rules don't currently provide a way to handle that:

```
#include <span>
#include <vector>

template <typename T> void f(const T&);
template <typename T> void g(const std::vector<T>&);
template <typename T> void h(std::span<T>);

void example()
{
    std::vector<int> v;

    f(v); // OK, T deduced as std::vector<int>
    g(v); // OK, T deduced as int
    h(v); // error: unable to deduce T
}
```

In the above example, based on our knowledge of `std::span`, it is easy to see that a match for the call to `h` is possible; if `T` were to somehow be deduced as `int`, then the call to `h` would be valid. And, in fact, there is an existing context in the language where that deduction succeeds:

```
#include <span>
#include <vector>

std::vector<int> v;
std::span s = v; // OK, s is std::span<int, std::dynamic_extent>
```

By applying this mechanism in the context of a function call, we can reproduce the successful deduction, and thereby make the language more useful and more self-consistent.

§ 3 Design

§ 3.1 General approach: CTAD behind the scenes

The mechanism is class template argument deduction (CTAD, [P0091R3]), initially introduced in C++17 with support for class templates, and later expanded in C++20 with support for alias templates ([P1814R0]). With CTAD, an unspecialized template name is used in a declaration as a placeholder for a specialization of that template, which is inferred from the declaration's initializer. A placeholder may appear in variable declarations, new expressions, function-style casts, and non-type template parameter declarations, but not in function parameter declarations.

Note that this paper *does not* propose allowing placeholders in function parameter declarations:

```
void f(std::pair p); // NOT proposed
```

Such a feature is certainly achievable, and may be useful in its own right, but this approach has limitations because the placeholder syntax disallows template arguments on the placeholder:

- It is impossible to express relationships between function parameters making use of the same template parameters from the function template. For instance, given the hypothetical declaration

```
void copy(std::span from, std::span to);
```

there is no straightforward way to establish that the value types of `from` and `to` are the same, or that the element type of `from` is `const`-qualified while the element type of `to` is not.

- CTAD using placeholders deduces arguments for *all* parameters of the template; there is no way to provide explicit arguments for some template parameters and deduce the others. Given a declaration

```
template <typename V> void f(std::tuple<std::string_view, V>);
```

we would like to be able to deduce `V` given an argument of type `std::pair<std::string, int>`.

Given these shortcomings and a desire for the feature to work with existing templates, the proposal is to make no changes to the existing syntax for function parameters. Instead, the approach is to generate an alias template from the function parameter type, and then perform template argument deduction for the initialization of a variable declared with the name of the alias template as its type and the function argument as its initializer.

For example, given the function declaration and call argument from the second bullet point above:

```
template <typename V>
void f(std::tuple<std::string_view, V>);

void example()
{
    std::pair<std::string, int> p = { "hello", 5 };
    f(p);
}
```

...an alias template and corresponding variable declaration are considered. Template arguments for the type of the variable are determined according to the rules for CTAD for alias templates ([P1814R0]):

```
// Hypothetical alias template
template <typename V>
using A = std::tuple<std::string_view, V>;
```

```
A x = p; // A deduced as A<int> (i.e. std::tuple<std::string_view, int>)
```

and the result is substituted back into the type of the function parameter:

```
f(p); // OK, V deduced as int
```

§ 3.2 Limitations of deduction guides

Consider this simple function template for copying data between two contiguous ranges:

```
template <typename T>
void copy(std::span<const T> from, std::span<T> to);
```

With the approach outlined above, this function template is now directly callable using spans, vectors, arrays (both built-in and `std::`), or even braced pointer/size pairs. (Of course, a more generic range concept could also have handled all of those cases and more, but this approach has the advantage of generating fewer template instantiations; everything converts to some flavor of span before the function call.) All of that would have required additional overloads without this feature.

But there's still a surprising gap in functionality here:

```
void example(std::span<const int> src1, std::span<int> src2)
{
    int dst[5];

    copy(src1, dst); // OK
    copy(src2, dst); // error: deduction failed for T from src2
}
```

This stems from an existing limitation with deduction guides; the same problem arises in existing code today:

```
#include <span>

template <typename ElementType, std::size_t Extent = std::dynamic_extent>
using const_span = std::span<const ElementType, Extent>;

std::span<int> x;
const_span<int> y = x; // OK
const_span z = x; // error: No guide for const_span from std::span<int,
                  std::dynamic_extent>
```

To understand why this fails, we have to examine the set of guides that are generated for `const_span`. The guides for `const_span` are generated from the guides for `std::span`, of which there are two that could conceivably be relevant to this situation:

```
namespace std
{
    // Guide generated from the copy constructor of std::span
    template <typename ElementType, size_t Extent>
        span(const span<ElementType, Extent>&) -> span<ElementType, Extent>;
}
```

```

// User-declared deduction guide for adapting contiguous ranges
template <ranges::contiguous_range R>
    span(R&&) -> span<remove_reference_t<ranges::range_reference_t<R>>>;
}

```

These are the corresponding guides generated for `const_span`:

```

template <typename ElementType, std::size_t Extent>
    const_span(const std::span<const ElementType, Extent>&) ->
        std::span<const ElementType, Extent>
    requires deduces-const-span<std::span<const ElementType, Extent>>;

template <std::ranges::contiguous_range R>
    const_span(R&&) ->
        std::span<std::remove_reference_t<std::ranges::range_reference_t<R>>>
            requires deduces-const-span<std::span<std::remove_reference_t<std::ranges::range_reference_t<R>>>>;

```

Here, *deduces-const-span* is a constraint that is satisfied if and only if template arguments for `const_span` are deducible from the type argument. This validates that, for a given set of template arguments, the return type of the guide can be expressed as a specialization of `const_span`.

The first guide fails because it's expecting a span of `const ElementType`, while `x` is a span of (non-const) `int`.

The second guide fails because the return type fails the deducibility constraint. Substituting the type of `x` for `R`, we get:

```

const_span(std::span<int>&) -> std::span<int>
    requires deduces-const-span<std::span<int>>;

```

where `std::span<int>` cannot be expressed as a specialization of `const_span`.

§ 3.2.1 Declaring deduction guides for alias templates

We could solve this by providing our own user-declared guides for `const_span`:

```

template <typename ElementType, std::size_t Extent>
    const_span(const std::span<ElementType, Extent>&) ->
        const_span<ElementType, Extent>;
// etc

```

...but the language does not currently allow us to do that. Users can only declare deduction guides for class templates, not alias templates.

This restriction is artificial. Language rules already require implementations to keep track of a set of guides for each alias template; allowing users to extend this set should pose no technical challenge. (Indeed, with Clang, this change required only removing a single check explicitly rejecting deduction guide declarations for alias templates.)

With the addition of user-declared deduction guides, we can rewrite the copy example in a form that works:

```
#include <span>

template <typename ElementType, std::size_t Extent = std::dynamic_extent>
using const_span = std::span<const ElementType, Extent>;

template <typename ElementType, std::size_t Extent>
    const_span(const std::span<ElementType, Extent>&) ->
    const_span<ElementType, Extent>;
template <std::ranges::contiguous_range R>
    const_span(R&&) ->
    const_span<std::remove_reference_t<std::ranges::range_reference_t<R>>>;
// etc

template <typename T>
void copy(const_span<T> from, std::span<T> to);

void example(std::span<const int> src1, std::span<int> src2)
{
    int dst[5];

    copy(src1, dst); // OK
    copy(src2, dst); // OK
}
```

§ 3.2.2 Alternative: Multiple return types for deduction guides

There is another way in which the copy example might be made to work without the need for a rewrite or a custom alias template with accompanying user-declared guides. Fundamentally, the problem is that a deduction guide can only map a given set of inputs to a single specialization of the template. Extending deduction guides to allow a list of return types would allow us to express deeper relationships between types and permit the implementation to choose the most appropriate alternative for a given context.

Continuing with the span example, we could add this user-declared deduction guide to show that spans of T are adaptable to spans of const T:

```
namespace std
{
    template <typename ElementType, size_t Extent>
    span(const span<ElementType, Extent>&)
        -> span<ElementType, Extent>,
           span<ElementType, dynamic_extent>,
           span<const ElementType, Extent>
           span<const ElementType, dynamic_extent>;
}
```

Each return type is an alternative deduction, expressed in decreasing order of preference. Variable declarations using the name of a class template would be unaffected by this change; the first alternative would always be chosen. These alternatives would come into

play for alias templates; the compiler would try each alternative in the order given, and the first to pass the deducibility constraint would be selected.

That would make the `const_span` example work without the need for additional user-declared deduction guides:

```
#include <span>

template <typename ElementType, std::size_t Extent = std::dynamic_extent>
using const_span = std::span<const ElementType, Extent>;

// No new deduction guides here!

std::span<int> x;
const_span<int> y = x; // OK
const_span z = x; // OK, implementation selects std::span<const int,
                  std::dynamic_extent>
```

This idea has not yet been fully examined and may be revisited in a future revision of this paper.

§ 3.3 Overload resolution

This feature makes some overloads viable that were not viable before, potentially expanding the set of viable functions for a given function call. This has the potential to make ambiguous a function call that was previously well-formed:

```
template <typename T>
struct X {};

template <typename T>
struct Y
{
    Y(X<T>);
};

template <typename T>
void f(X<T>, float); // #1

template <typename T>
void f(Y<T>, int); // #2

f(X<short>(), 5); // OK before, ambiguous with this proposal
```

Without this proposal, the call to `f` selects `#1`; `#2` is non-viable because `T` cannot be deduced from the call. With this proposal, the call is ambiguous; `#2` is now viable with `T` deduced as `short`, but `#1` is a better match for the first argument, while `#2` is a better match for the second argument.

It is currently unknown how much code in the wild would be broken by adopting this feature. To avoid breaks, we could introduce a rule that makes `#2` a worse match than `#1` by virtue of its use of this feature. Such a rule would be decidedly unprincipled, and may

lead to some surprising results; arguably the call to *f* *should* be ambiguous. Still, even with the compatibility work-around, the feature should prove useful.

§ 4 Wording

All changes are presented relative to [N4986].

Modify §12.2.2.9 [over.match.class.deduct] paragraph 1:

- 1 When resolving a placeholder for a deduced class type (9.2.9.8 [dcl.type.class.deduct]) where the *template-name* names a primary class template *C*, a set of functions and function templates, called the guides of *C*, is formed comprising:
- [...]
 - (1.4) — For each *deduction-guide* declared for *C* (13.7.2.3 [temp.deduct.guide]), a function or function template with the following properties: as described below.
 - (1.4.1) — The *template-head*, if any, and *parameter-declaration-clause* are those of the *deduction-guide*.
 - (1.4.2) — The return type is the *simple-template-id* of the *deduction-guide*.
- [...]

Modify §12.2.2.9 [over.match.class.deduct] paragraph 3:

- 3 When resolving a placeholder for a deduced class type (9.2.9.3 [dcl.type.simple]) where the *template-name* names an alias template *A*, the *defining-type-id* of *A* must be of the form
- $$\text{typename}_{opt} \text{nested-name-specifier}_{opt} \text{template}_{opt} \text{simple-template-id}$$
- as specified in 9.2.9.3 [dcl.type.simple]. Let *B* be the template named by the *simple-template-id* of the *defining-type-id* of *A*. The guides of *A* are the set of functions or function templates formed as follows: comprising:
- (3.1) — For each function or function template *f* in the guides of the template named by the *simple-template-id* of the *defining-type-id* *B*, a corresponding function or function template *f'* formed as follows: the template *Template* arguments of for the template parameters used in the return type of *f* are deduced from the *defining-type-id* of *A* according to the process in 13.10.3.6 [temp.deduct.type] with the exception that deduction does not fail if not all template arguments are

deduced. If deduction fails for another reason, proceed with an empty set of deduced template arguments. Let *g* denote the result of substituting these deductions into *f*. If substitution ~~succeeds~~ fails, there is no *f'* corresponding to this *f*; otherwise, form a function or function template *f'* with has the following properties and add it to the set of guides of *A*:

— [...]]

- (3.2) — For each *deduction-guide* declared for *A* (13.7.2.3 [temp.deduct.guide]), a function or function template as described below.

If a function or function template in the guides of *A* formed from the guides of *B* would potentially conflict (6.4.1 [basic.scope.scope]) with a function or function template formed from a *deduction-guide* declared for *A* if given the same name, then the former is discarded.

Insert a new paragraph immediately following the above modified paragraph:

4 A guide formed from a *deduction-guide* has the following properties:

- (1) — The *template-head*, if any, and *parameter-declaration-clause* are those of the *deduction-guide*.
- (2) — The return type is the *simple-template-id* of the *deduction-guide*.

Modify §13.7.2.3 [temp.deduct.guide] paragraph 1:

deduction-guide:
*explicit-specifier*_{opt} *template-name* (*parameter-declaration-clause*) *requires-clause*_{opt} $\rightarrow$ *simple-template-id* ;

- 1 Deduction guides are used when a *template-name* appears as a type specifier for a deduced class type (9.2.9.8 [dcl.type.class.deduct]). A deduction guide is declared for the template named by its *template-name*. Deduction guides are not found by name lookup. Instead, when performing class template argument deduction (12.2.2.9 [over.match.class.deduct]), all reachable deduction guides declared for the ~~class~~ template are considered.

~~*deduction-guide*:
*explicit-specifier*_{opt} *template-name* (*parameter-declaration-clause*) *requires-clause*_{opt} $\rightarrow$ *simple-template-id* ;~~

Modify §13.7.2.3 [temp.deduct.guide] paragraph 3:

- 3 The same restrictions apply to the *parameter-declaration-clause* of a deduction guide as in a function declaration (9.3.4.6 [dcl.fct]), except that a generic parameter type placeholder (9.2.9.7 [dcl.spec.auto]) shall not appear in the *parameter-declaration-clause* of a deduction guide. The *simple-template-id* *template-name* shall name a ~~class~~ *template specialization* *deducible template* (9.2.9.3 [dcl.type.simple]) and the *simple-template-id* shall name a specialization of that template. The *template-name* shall be the same *identifier* as the *template-name* of the *simple-template-id*. A *deduction-guide* shall inhabit the scope to which the corresponding ~~class~~ template belongs and, for a member ~~class~~ template, have the same access. Two deduction guide declarations for the same ~~class~~ template shall not have equivalent *parameter-declaration-clauses* if either is reachable from the other.

Split §13.10.3.2 [temp.deduct.call] paragraph 1 after the first sentence, with modifications as shown:

- 1 Template argument deduction is done by comparing each function template parameter type ~~(call it P)~~ that contains *template-parameters* that participate in template argument deduction with the type *A* of the corresponding argument of the call ~~(call it A)~~ as described in 13.10.3.6 [temp.deduct.type], with potential modifications to P and A as described below.

Form a new paragraph from the remainder, with modifications as shown, splitting it again after the first example:

- 2 If removing references and cv-qualifiers from P gives `std::initializer_list<P'>` or `P'[N]` for some P' and N and the argument is a non-empty initializer list (9.4.5 [dcl.init.list]), then deduction is performed instead for each element of the initializer list independently, taking P' as separate function template parameter types P'_i and the *i*th initializer element as the corresponding argument. In the P'[N] case, if N is a non-type template parameter, N is deduced from the length of the initializer list. ~~Otherwise, an initializer list argument causes the parameter to be considered a non-deduced context (13.10.3.6 [temp.deduct.type]).~~

[Example:

```
template<class T> void f(std::initializer_list<T>);  
f({1,2,3}); // T deduced as int  
f({1,"asdf"}); // error: T deduced as both  
             int and const char*  
  
template<class T> void g(T);
```

```

g({1,2,3}); // error: no argument
deduced for T

template<class T, int N> void h(T const(&)[N]);
h({1,2,3}); // T deduced as int; N
deduced as 3

template<class T> void j(T const(&)[3]);
j({42}); // T deduced as int; array
bound not considered

struct Aggr { int i; int j; };
template<int N> void k(Aggr const(&)[N]);
k({1,2,3}); // error: deduction fails,
no conversion from int to Aggr
k({{1},{2},{3}}); // OK, N deduced as 3

template<int M, int N> void m(int const(&)[M][N]);
m({{1,2},{3,4}}); // M and N both deduced as
2

template<class T, int N> void n(T const(&)[N], T);
n({{1},{2},{3}},Aggr()); // OK, T is Aggr, N is 3

template<typename T, int N> void o(T (* const (&)[N])(T)) {
}
int f1(int);
int f4(int);
char f4(char);
o({ &f1, &f4 }); // OK, T
deduced as int from first element, nothing
// deduced
from second element, N deduced as 2
o({ &f1, static_cast<char(*)>(&f4) }); // error:
conflicting deductions for T

— end example ]

```

Insert a new paragraph immediately following the above modified paragraph:

- 3 If removing references and cv-qualifiers from P yields a specialization P' of a deducible template (9.2.9.3 [dcl.type.simple]) other than `std::initializer_list`, and either

- the argument is an initializer list, or
- the deducible template is a class template and A is neither a specialization of the same class template nor a class derived from such,

deduction is performed as described in 12.2.2.9 [over.match.class.deduct] for a declaration of the form

PA x = a;

where PA is the name of a hypothetical alias template whose *defining-type-id* is P' and whose template parameter list consists of all the template parameters of the enclosing function template that appear in P', and a is the function argument corresponding to P. If the deduction succeeds, then the type of x is used in place of A for type deduction.

[Example:

```
template<class T> struct P { P(T*); };  
template<class T> void p(P<T>);  
float x;  
p(&x); // OK, T deduced as float  
  
template<class T> struct V { V(std::initializer_list<T>);  
};  
template<class T> void q(V<T>);  
q({ 1, 2, 3 }); // OK, T deduced as int  
  
template<class T1, class T2> struct Pair { T1 a; T2 b; };  
template<class T1, class T2> r(const Pair<T1, T2>&);  
r({ "pi", 3.14159 }); // T1 deduced as const  
char*; T2 deduced as double  
  
template<class T, std::size_t N> struct A { A(T (&)[N]); };  
template<std::size_t N> s(A<int, N>&&);  
int arr[] = { 1, 2, 3, 4, 5 };  
s(arr); // OK, N deduced as 5
```

— end example]

Insert a new paragraph immediately following the above inserted paragraph:

- 4 In all other cases, an initializer list argument causes the parameter to be considered a non-deduced context (13.10.3.6 [\[temp.deduct.type\]](#)).

[Example:

```
template<class T> void g(T);  
g({1,2,3}); // error: no argument  
deduced for T
```

— end example]

Retain the remainder of the original paragraph 1 unchanged as a new paragraph immediately following the above inserted paragraph:

- 5 For a function parameter pack that occurs at the end of the *parameter-declaration-list*, deduction is performed for each remaining argument of the call, taking the type P of the *declarator-id* of the function parameter pack as the corresponding function template parameter type. Each

deduction deduces template arguments for subsequent positions in the template parameter packs expanded by the function parameter pack. When a function parameter pack appears in a non-deduced context (13.10.3.6 [temp.deduct.type]), the type of that pack is never deduced.

[*Example:*

```
template<class ... Types> void f(Types& ...);
template<class T1, class ... Types> void g(T1, Types ...);
template<class T1, class ... Types> void g1(Types ..., T1);

void h(int x, float& y) {
    const int z = x;
    f(x, y, z);                // Types deduced as int,
                               // float, const int
    g(x, y, z);                // T1 deduced as int; Types
                               // deduced as float, int
    g1(x, y, z);               // error: Types is not
                               // deduced
    g1<int, int, int>(x, y, z); // OK, no deduction occurs
}
```

— *end example*]

§ 5 Acknowledgments

Thanks to Mike Spertus for some early feedback on the proposal.

§ 6 References

[N4986] Thomas Köppe. 2024-07-16. Working Draft, Programming Languages — C++.

<https://wg21.link/n4986>

[P0091R3] Mike Spertus, Faisal Vali, Richard Smith. 2016-06-24. Template argument deduction for class templates (Rev. 6).

<https://wg21.link/p0091r3>

[P1814R0] Mike Spertus. 2019-07-28. Wording for Class Template Argument Deduction for Alias Templates.

<https://wg21.link/p1814r0>